Reflection

Trent Gwathney | IS601 | Module 10

Adding users to the calculator helped me understand how database models and API schemas serve different purposes. SQLAlchemy defines stored fields and enforces unique usernames and email addresses. Pydantic checks incoming requests, while a separate response schema controls what the API returns. That separation matters because a valid user record contains a password hash that should never appear in a public response.

Password handling was another useful lesson. Argon2id produces a salted hash instead of storing the original password. Testing both correct and incorrect passwords helped me understand verification, and checking that identical passwords produce different hashes made the role of salts clearer.

One challenge was extending the project without disrupting the previous assignment's database records. An Alembic migration created the new user table in a separate calculator schema, preserving the earlier tables. Real PostgreSQL tests also needed isolation. Transactions and savepoints allowed tests to exercise database constraints and roll back their changes afterward. Windows had also reserved port 8000, so I configured the local application to use port 18000.

The image scan exposed another challenge: passing application tests did not mean the container's operating system packages were free of known vulnerabilities. Switching to an updated Alpine base resolved those findings. The workflow scans before publishing, and pulling the published image adds a separate API check. I feel more confident with validation, password hashing, and database tests. I still want more practice writing migrations for existing production data, interpreting vulnerability reports, and adding authentication and authorization.